



S⁴D: Style and Structure Similarity for Site Domains

S⁴D: Style and Structure Similarity for Site Domains

Panagiotis Favvatas

Bachelor Thesis, Department of Informatics And Telecommunications
University of the Peloponnese, July 2025

Copyright © 2025 University of Peloponnese. All Rights Reserved

S⁴D: Style and Structure Similarity for Site Domains

Panagiotis Favvatas

A thesis submitted in partial fulfillment
of the requirements for the degree of
Informatics and Telecommunications

Committee

Professor, Spiros Skiadopoulos, Supervisor

Professor, [Committee Member 1]

Associate Professor, [Committee Member 2]

University of the Peloponnese

2025

Περίληψη

Τα τελευταία χρόνια, οι ιστότοποι έχουν γίνει πιο πολύπλοκοι και ποικιλόμορφοι ως προς τον σχεδιασμό και τη δομή τους. Παρά την ποικιλομορφία αυτή, οι ιστότοποι που βρίσκονται στον ίδιο θεματικό τομέα ή εξυπηρετούν παρόμοιους σκοπούς συχνά μοιράζονται κοινά παρουσιαστικά και δομικά μοτίβα. Η αναγνώριση και η σύγκριση αυτών των μοτίβων σε διαφορετικούς τομείς ιστότοπων είναι πολύτιμη για την επίτευξη αποτελεσμάτων ως προς την ταξινόμηση ιστοσελίδων, την αυτοματοποιημένη ανάλυση διεπαφών χρήστη (UIs) και την αποτύπωση των θεματικών τους τομέων. Ωστόσο, η δομή και η οπτική παρουσίαση των περιεχομένων ιστού είναι συχνά μη τυποποιημένη και ασυνεπής, αυξάνοντας τη δυσκολία τέτοιων αναλύσεων. Σε αυτήν την πτυχιακή εργασία, παρουσιάζεται το **S⁴D**, ένα σύστημα που αυτοματοποιεί τη διαδικασία εξαγωγής, μοντελοποίησης και σύγκρισης των παρουσιαστικών και δομικών ομοιοτήτων διαφορετικών ιστοτόπων. Το σύστημα αναλύει ένα σύνολο ιστοσελίδων μέσω τεχνικών αυτοματοποιημένης περιήγησης, μοντελοποιώντας τα CSS χαρακτηριστικά των στοιχείων τους και οργανώνοντάς τα με τρόπο που επιτρέπει την περαιτέρω ανάλυση. Συγκεκριμένα, μετατρέποντας τα στοιχεία σελίδας σε διανυσματικές αναπαραστάσεις και εφαρμόζοντας τεχνικές μη-επιβλεπόμενης συσταδοποίησης, το σύστημα εντοπίζει ουσιαστικές συστάδες των στοιχείων βάσει των οπτικών τους χαρακτηριστικών. Χρησιμοποιώντας αυτήν την συσταδοποιημένη αναπαράσταση, το σύστημα μοντελοποιεί τα παραγόμενα δεδομένα για κάθε ιστότοπο, εξάγει μοτίβα διάταξης στοιχείων και υπολογίζει την ομοιότητα μεταξύ τους χρησιμοποιώντας διαφορετικές μετρικές απόστασης. Τέλος, το σύστημα παρουσιάζει έναν πίνακα ομοιότητας μεταξύ των ιστοτόπων, προσφέροντας πληροφορίες σχετικά με τη δομική και παρουσιαστική ομοιότητα των ιστοσελίδων τους.

Abstract

In recent years, websites have become increasingly complex and diverse in design and structure. Despite this diversity, websites belonging to the same domain or serving similar purposes frequently exhibit similar stylistic and structural characteristics. Identifying and comparing these patterns across different site domains is valuable for tasks such as web classification, automated user interface (UI) analysis, and domain topic labeling. However, the structure and visual presentation of web content is often unstandardized and inconsistent, making such analysis a challenging task. This thesis introduces **S⁴D**, a system that automates the process of extracting, modeling, and comparing the style and structure of websites across different domains. The system parses a set of webpages using browser automation techniques, modeling CSS-related properties of web elements, and structuring them in a way that enables further analysis. By transforming page elements into vector representations and applying unsupervised clustering techniques, the system identifies meaningful groupings of visual and structural components. Using this clustered representation, the system exploits the resulting data modeling of each domain, extracting high-level patterns of element arrangement, and computes the inter-domain similarity using multiple distance metrics. Finally, the system presents a similarity matrix between domains, providing insight into the structural and stylistic resemblance of their webpages.

Acknowledgements

I would like to thank Professor Spiros Skiadopoulos for the valuable scientific guidance he has provided me throughout the course of this thesis, as well as for the immediate help he offered me whenever it was needed. In addition, I would like to thank the professors of the Department of Informatics and Telecommunications, who have given me the knowledge needed in order to further develop myself and contribute to the advancement of this scientific field.

Furthermore, I would like to express my sincere gratitude to a colleague of mine who supported me significantly during the preparation of this thesis.

Finally, I would like to thank my family and my friends for their continuous support and encouragement throughout all these years.

Contents

Chapter 1

Introduction

This thesis presents **S⁴D**, a system for analyzing and detecting similarities between websites. **S⁴D** is a multi-layered system that aims to facilitate the modeling and analysis of websites' visual and structural similarity. Through a variety of functionalities and a Graphical User Interface (GUI), it is able to yield meaningful insight into the websites' visual and structural similarity analysis process, ultimately generating a similarity matrix for all input webpages.

1.1 Problem Statement

The current landscape of web analysis tools focuses primarily on content-based analysis, often overlooking the significance of visual design patterns and structural similarities between websites. In the era of rapidly expanding web content, understanding the structural similarity between websites has become increasingly important for various applications including web mining, content analysis, and automated web processing. However, traditional approaches face several critical challenges.

1.1.1 Web Structure Analysis Challenges

Modern web structure analysis presents several unique technical challenges that our S4D system specifically addresses.

1.1.1.1 DOM Complexity and Recursive Processing

Modern websites contain deeply nested DOM structures with thousands of elements requiring recursive traversal algorithms. Our system handles complex parent-child relationships through unique ID generation and hierarchical tree processing, managing elements at multiple depth levels (1–10) while maintaining structural integrity during analysis.

1.1.1.2 CSS Style Diversity and Standardization

The variety of CSS styling approaches across websites creates inconsistency in structural representation. Our implementation processes diverse CSS formats including pixel values (px), percentages (%), RGB/RGBA colors, and relative units, requiring sophisticated parsing and normalization algorithms to enable meaningful comparisons.

1.1.1.3 Attribute Extraction and Feature Engineering

Extracting meaningful features from HTML elements involves processing hundreds of CSS properties per element. Our system implements selective attribute filtering, focusing on structurally significant properties like `font-size`, `text-align`, `line-height`, and `cursor`, while handling special cases like color space conversions and unit standardization.

1.1.1.4 Scalability and Performance Optimization

Processing large numbers of websites requires efficient algorithms and memory management. Our system implements domain-based processing modes, ChromeDriver optimization, and parallel data processing to handle batch website analysis while minimizing computational overhead and ensuring reliable performance across different scales.

1.1.1.5 Multi-dimensional Similarity Computation

Defining meaningful similarity measures for web structures involves multiple algorithmic challenges. Our approach combines HDBSCAN clustering with DBCV (Density-Based Cluster Validation) scoring, cosine similarity matrices, and role vector analysis to capture structural, visual, and positional relationships between website elements.

1.1.1.6 Clustering Parameter Optimization

Achieving optimal clustering results requires dynamic parameter tuning. Our system implements grid search optimization over cluster size ranges and selection epsilon values, with automatic outlier reassignment to nearest cluster centroids based on Euclidean distance calculations.

1.1.1.7 Cross-Domain Pattern Recognition

Identifying design trends across different domains requires sophisticated comparison frameworks. Our implementation uses domain-level aggregation, transition matrices between design elements, and combined similarity metrics (cosine, EMD, Hausdorff) to enable meaningful cross-domain analysis.

1.1.2 Research Motivation

The need for automated web structural analysis stems from several key factors:

- **Content Management:** Organizations need tools to understand and manage large collections of web content based on structural and visual characteristics.
- **Design Pattern Analysis:** Web developers and designers benefit from understanding common structural patterns and visual trends across websites in their domain.
- **Accessibility Evaluation:** Automated structural analysis can help identify accessibility issues and design inconsistencies across multiple web properties.
- **Web Analytics Enhancement:** Structural similarity analysis provides deeper insights into website categorization and user experience patterns beyond traditional content-based metrics.

1.2 Research Objectives

The primary objectives of this research are:

1. To develop a robust system for extracting and analyzing structural and visual features from websites
2. To create effective similarity metrics that can accurately measure relationships between different websites
3. To implement clustering algorithms that can group websites based on their design and structural characteristics
4. To provide a comprehensive comparison framework for website analysis across different domains
5. To validate the effectiveness of the proposed system through extensive testing and evaluation

1.3 Contributions

This thesis makes the following key contributions:

- **Novel feature extraction methodology:** Development of comprehensive techniques for extracting both structural and visual features from websites
- **Advanced similarity metrics:** Introduction of new metrics specifically designed for website comparison
- **Integrated clustering approach:** Implementation of clustering algorithms optimized for website analysis
- **Comprehensive evaluation framework:** Establishment of evaluation metrics and datasets for assessing website similarity systems
- **Practical applications:** Demonstration of real-world applications in web design analysis and automated categorization

1.4 Thesis Structure

The remainder of this thesis is organized as follows:

- **Chapter 2: Literature Review** presents a comprehensive review of existing approaches to website analysis, similarity measurement, and clustering techniques relevant to web data.
- **Chapter 3: Methodology** describes the proposed S⁴D system architecture, including feature extraction methods, similarity metrics, and clustering algorithms.
- **Chapter 4: Implementation** details the technical implementation of the system, including data structures, algorithms, and software architecture decisions.
- **Chapter 5: Experimental Design** outlines the experimental methodology, datasets used, and evaluation metrics employed to assess the system's performance.
- **Chapter 6: Results and Analysis** presents the experimental results, performance analysis, and comparison with existing approaches.

- **Chapter 7: Discussion** provides an in-depth discussion of the findings, limitations, and implications of the research.
- **Chapter 8: Conclusion and Future Work** summarizes the key findings, contributions, and suggests directions for future research.

Chapter 2

Related Work

This chapter reviews research related to the structural and visual analysis of web pages, highlighting prior work in web parsing, information extraction, and representation learning. The discussion emphasizes how these approaches inform, contrast with, and complement the S⁴D system. The review is structured into three major areas: (1) structural understanding of web documents, (2) information extraction and representation approaches, and (3) similarity and clustering methods for web analysis. Finally, we compare S⁴D with prior systems to highlight its unique contributions.

2.1 Structural Understanding of Web Documents

One of the earliest challenges in web analysis has been the ability to combine textual semantics with structural information. Chen et al. [1] introduced **WebSRC**, a large-scale dataset for structural reading comprehension. Their work demonstrates that effective web understanding requires not only semantic interpretation but also the hierarchical organization of HTML elements. WebSRC provides 400K question–answer pairs across 6.4K web pages, combining HTML source code, screenshots, and metadata. While WebSRC focuses on question answering, its emphasis on DOM-aware structural comprehension is relevant to S⁴D’s structural modeling. Unlike WebSRC, however, S⁴D does not address query answering but instead targets structural and stylistic similarity across domains.

Building on this idea of structural representation, Deng et al. [2] proposed **DOM-LM**, a transformer-based model that encodes both textual content and DOM structure to learn generalizable representations of web pages. Their approach addresses the shortcomings of methods that rely solely on visual rendering or plain-text representations. DOM-LM achieves strong generalization in few-shot and zero-shot tasks such as attribute extraction and information retrieval. In contrast, S⁴D applies structural encoding not for semantic extraction but to identify recurring patterns of element arrangement across domains, enabling inter-domain similarity assessment.

2.2 Information Extraction and Representation

The extraction of structured information from web pages has been another central research theme. Gulhane et al. [3] developed **Vertex**, a wrapper induction system deployed at Yahoo! for large-scale record extraction. Vertex introduced algorithms for grouping similar pages, generating XPath-based extraction rules, and adapting to structural changes. It achieved web-scale deployment, extracting over 250 million records from more than 200 websites. Vertex demonstrates the scalability of web information extraction but is tailored to structured record retrieval. By contrast, S⁴D does not generate wrappers or extract database-like content; it focuses instead on comparing visual and structural layouts across entire domains.

More recently, Lockard et al. [1] proposed **ZeroShotCeres**, which tackles the challenge of zero-shot relation extraction from semi-structured web pages. Their model employs graph neural networks to capture both textual semantics and visual cues such as layout, font size, and color. This enables generalization to previously unseen templates without the need for template-specific training data. ZeroShotCeres highlights the importance of structural and visual cues for generalizable web analysis. While their focus is relation extraction, S⁴D instead leverages structural and visual properties to compute domain-level similarity, not entity-level relations.

2.3 Similarity and Clustering in Web Analysis

Similarity measurement and clustering techniques have been applied in web analysis to group documents, identify templates, and extract structure. Traditional approaches often prioritize textual similarity, but recent advances emphasize structural and visual dimensions. For example, graph-based models such as ZeroShotCeres [1] highlight the utility of representing web pages as graphs where nodes correspond to DOM elements and edges represent structural or semantic relationships.

The S⁴D system extends these ideas by combining multiple similarity dimensions—structural, visual, positional, and semantic—and introducing role vector similarity to capture the functional roles of elements within a document. For clustering, S⁴D adopts HDBSCAN [2], which automatically determines the number of clusters, handles noise, and reveals hierarchical patterns. This makes it particularly suited to web data, which is both high-dimensional and heterogeneous. Unlike prior work, S⁴D applies clustering not at the level of individual pages but across domains, enabling the analysis of cross-domain stylistic and structural similarity.

2.4 Comparison with Prior Work

The reviewed works demonstrate significant progress in web analysis, yet they address different objectives from S⁴D:

- **WebSRC [3]**: Focuses on question answering over structured web data, highlighting the need for structural comprehension. S⁴D instead uses structure for cross-domain similarity analysis.
- **DOM-LM [4]**: Learns generalizable document embeddings from DOM+text representations. S⁴D leverages DOM+CSS for unsupervised clustering rather than representation learning.
- **Vertex [5]**: Extracts structured records at web scale via wrapper induction. S⁴D does not extract records but compares layout and style across domains.
- **ZeroShotCeres [1]**: Performs zero-shot relation extraction using structural and visual features. S⁴D uses similar structural/visual cues but for similarity computation and domain-level clustering.

2.5 Research Gaps and Opportunities

The literature reveals several research gaps that S⁴D addresses:

- **Cross-domain similarity**: None of the reviewed systems compute similarity matrices across domains, which is central to S⁴D.
- **Multi-dimensional analysis**: Existing work typically emphasizes content or relations; S⁴D integrates structural, visual, and positional dimensions.

- **Clustering across stylistic patterns:** Prior systems rarely apply clustering to identify design-level commonalities across domains. S⁴D introduces a multi-level clustering approach for this purpose.
- **Scalability to modern web design:** Systems like Vertex demonstrate scale for extraction tasks, but scalable cross-domain similarity analysis remains underexplored.

2.6 Summary

This chapter has reviewed four representative systems—WebSRC, DOM-LM, Vertex, and ZeroShotCeres—that address different aspects of structural web understanding, information extraction, and similarity-based analysis. Each system contributes important ideas relevant to S⁴D, yet none directly targets cross-domain visual and structural similarity measurement via unsupervised clustering. The gaps identified above motivate the design choices presented in the following chapter.

Chapter 3

The S⁴D System

This chapter presents the complete S⁴D (Style and Structure Similarity for Site Domains) system, detailing the 7-step workflow from web page acquisition through similarity analysis to final result presentation. The system combines advanced web parsing, clustering algorithms, and multi-dimensional similarity computation to provide robust website comparison and categorisation capabilities.

3.1 System Overview

The S⁴D methodology employs a comprehensive multi-stage pipeline that transforms raw web pages into structured similarity matrices and clustering results. The system architecture consists of seven main processing components:

1. **Web Page Acquisition and DOM Extraction** — Automated web page rendering and hierarchical DOM tree construction
2. **Multi-Level CSS Property Analysis** — Comprehensive CSS attribute extraction and normalisation across multiple DOM depth levels
3. **Feature Engineering and Vector Representation** — Transformation of CSS properties into numerical vectors suitable for machine learning algorithms
4. **Advanced Clustering Analysis** — HDBSCAN-based clustering with automatic parameter optimisation and quality validation
5. **Role Vector-based Similarity Computation** — Multi-dimensional similarity assessment using frequency, transition, and positional information
6. **Multi-Metric Similarity Framework** — Combined similarity scoring using geometric mean of multiple distance metrics
7. **Result Presentation** — Interactive HTML generation and similarity matrix display

3.2 Technology Stack

The S⁴D system is built using a technology stack that ensures scalability, performance, and maintainability across all system components.

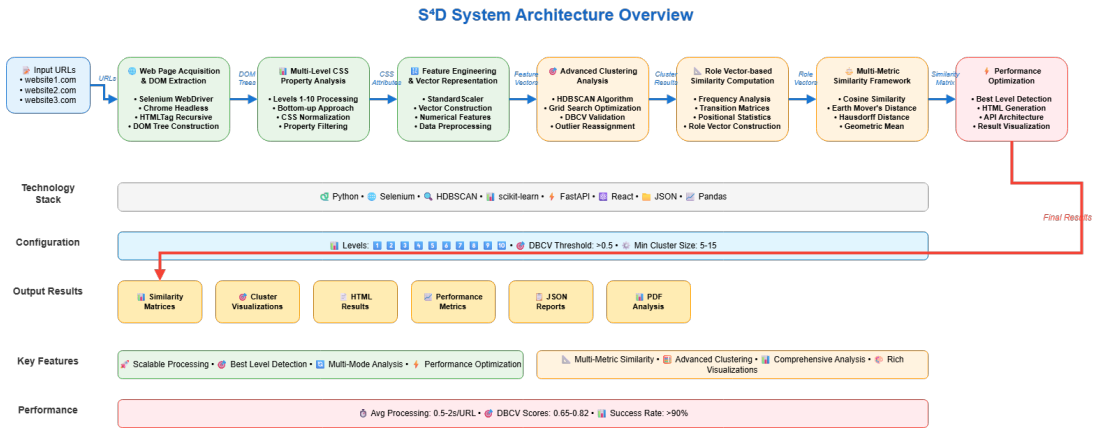


Figure 3.1: S⁴D System Architecture Overview showing the seven main components and data flow from web page acquisition through similarity computation to final clustering results.

3.2.1 Backend Technologies

Core Programming Language

- **Python 3.8+:** Primary backend development language with a rich ecosystem for data science and web development.
- **Flask:** Lightweight REST API framework for Python web services, providing HTTP endpoints for URL processing and result retrieval.
- **Selenium WebDriver 4.x:** Web automation framework for DOM extraction and CSS analysis using a headless Chrome browser.

Data Processing Libraries

- **NumPy:** Numerical computing and array operations for vector calculations.
- **Pandas:** Data manipulation and analysis for role vector computation.
- **Scikit-learn:** Machine learning algorithms and preprocessing, including `StandardScaler` and `cosine_similarity`.

Clustering and Similarity Analysis

- **HDBSCAN:** Hierarchical density-based clustering with automatic cluster detection ?.
- **DBCV:** Density-Based Cluster Validation for clustering quality assessment ?.
- **SciPy:** Scientific computing for distance calculations (`cdist`) and statistical functions.

3.2.2 Frontend and Infrastructure

- **React 18.x:** Component-based frontend framework providing the interactive user interface (port 3000).
- **Node.js static server:** A lightweight Node.js HTTP server that serves the generated HTML result files (port 5005).
- **mprocs:** Multi-process orchestration tool that starts all three services (Flask API, React frontend, Node.js file server) with a single command.
- **JSON:** Structured data storage and exchange format used throughout the system.

3.3 Web Parsing

This section details the web parsing implementation that combines automated DOM extraction with multi-level structural analysis.

3.3.1 Automated Web Rendering

The S⁴D system employs Selenium WebDriver with Chrome browser in headless mode to ensure consistent web page rendering. The browser is configured with:

- **Headless mode** (`-headless`): eliminates GUI overhead and ensures reproducible rendering.
- **Fixed viewport**: 1920×1080 pixels, ensuring consistent element positioning across pages.
- **Sandbox flags** (`-no-sandbox`, `-disable-dev-shm-usage`): enable execution in server environments.

3.3.2 Hierarchical DOM Tree Construction

The DOM extraction algorithm employs recursive traversal through the `HTMLTag` class. Each web element is processed as follows.

3.3.2.1 Unique Element Identification

A unique ID is generated for each element by combining a millisecond timestamp with a random integer:

$$\text{unique_id} = \text{timestamp}_{ms} \times 10^{10} + \text{random}_{[10^9, 10^{10})} \quad (3.1)$$

3.3.2.2 Comprehensive Attribute Extraction

Three categories of information are extracted per element:

1. **HTML attributes**: obtained via JavaScript execution in the browser context.
2. **Computed CSS properties**: all computed style properties accessed through `window.getComputedStyle()`.
3. **Geometric properties**: element bounding rectangle coordinates via `getBoundingClientRect()`.

3.3.2.3 Text Content Processing

Text content is extracted via direct DOM node traversal in JavaScript, isolating only text nodes that are direct children of the element:

```

1 let textContent = '';
2 for (let node of element.childNodes) {
3   if (node.nodeType === Node.TEXT_NODE) {
4     textContent += node.textContent;
5   }
6 }
7 return textContent.trim();

```

Listing 3.1: Text Content Extraction

The extracted text is then normalised by removing non-printable ASCII characters while preserving Unicode.

3.3.3 Multi-Level DOM Analysis

The system implements a bottom-up hierarchical analysis across 10 configurable levels (levels 1–10). The level represents the path length from a leaf node upward through the DOM tree:

- **Level 1:** Leaf elements (bottom-most nodes with no children).
- **Level 2:** Leaf elements plus their immediate parent.
- **Level 3:** Leaf elements plus parent and grandparent.
- **Levels 4–10:** Extended upward paths capturing broader structural context.

This bottom-up approach ensures the most content-specific elements are captured first, with increasing levels incorporating wider structural context.

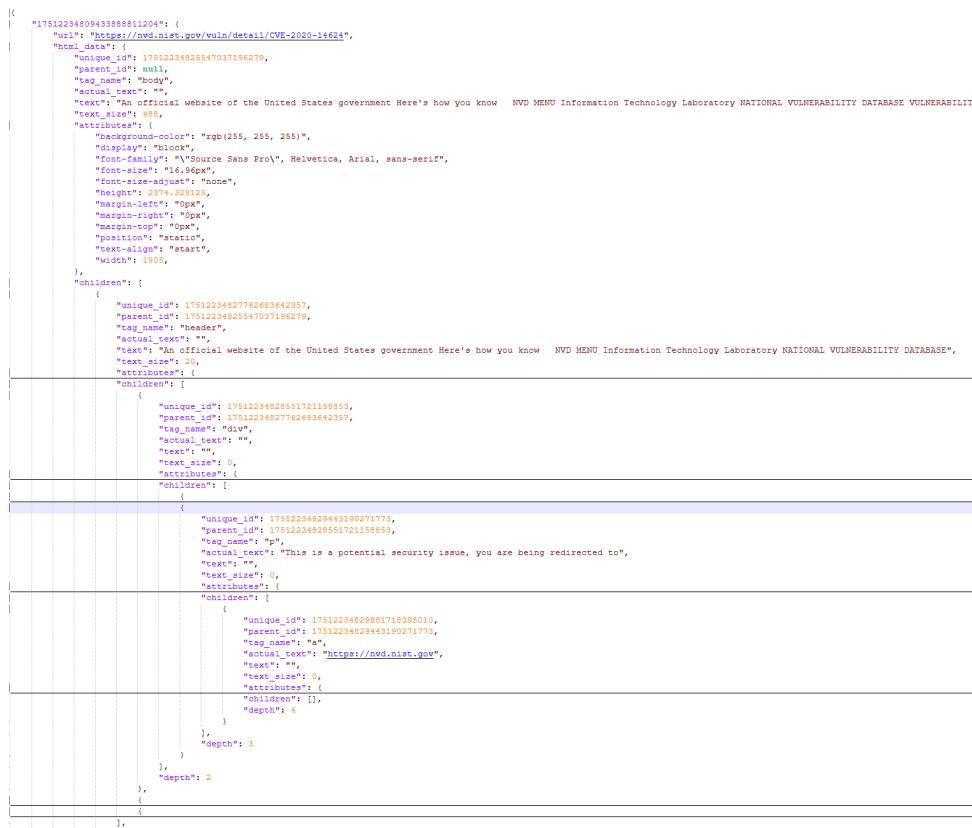


Figure 3.2: Bottom-up DOM Tree Traversal: Level 1 captures leaf nodes; higher levels extend the path toward the root.

3.3.4 CSS Property Analysis and Normalisation

3.3.4.1 Attribute Selection

The system focuses on a curated set of CSS properties with discriminative power for structural and visual similarity. These include typography properties (**font-size**, **font-weight**, **font-style**, **color**), layout properties (**text-align**, **line-height**, **margin**, **padding**, **border** properties), and decoration properties (**text-decoration-style**, **text-decoration-color**). Properties such as positioning (**display**, **position**) and animations are intentionally excluded as they introduce noise for cross-domain structural comparison.

3.3.4.2 CSS Value Normalisation

The system applies consistent normalisation rules:

Pixel values:

$$\text{normalised} = \begin{cases} -1 & \text{if value = "auto"} \\ -\frac{\text{percentage}}{100} & \text{if value contains "\%"} \\ \text{float}(\text{value.replace("px", "")}) & \text{otherwise} \end{cases} \quad (3.2)$$

Colour space conversion: RGB/RGBA values are converted to HSV for perceptual uniformity using Python’s `colorsys.rgb_to_hsv()`.

HSV Colour Space Conversion

```
1 def extract_hsv_values(color_str):
2     rgb = tuple(map(int, re.findall(r'\d+', color_str)))
3     h, s, v = colorsys.rgb_to_hsv(
4         rgb[0]/255.0, rgb[1]/255.0, rgb[2]/255.0)
5     return h, s, v
```

Dual-coordinate properties (margins, padding with two values “Xpx Ypx”) are ordered so that $X \geq Y$ always holds; if not, both values are negated. This ensures a canonical representation for symmetric and asymmetric spacing.

3.4 Data Modelling

3.4.1 Feature Vector Construction

For each DOM element at a given level, the system constructs a feature vector by concatenating the normalised values of all selected CSS attributes. Non-numeric CSS values (e.g., `left`, `bold`) are mapped to integers via a global string-to-integer vocabulary (`str_VSM`), with special handling for framework aliases (e.g., Bootstrap’s `start` $\rightarrow$ `left`).

All feature vectors are standardised using `StandardScaler`:

$$X_{\text{scaled}} = \frac{X - \mu}{\sigma} \quad (3.3)$$

3.4.2 HDBSCAN Clustering Integration

3.4.2.1 Parameter Optimisation

The system implements a grid search over HDBSCAN parameters:

- **Minimum cluster size:** ranges from 5% to 10% of the total number of elements.
- **Selection epsilon:** ranges from 0.0 to 1.0 in steps of 0.333.
- **Early stopping:** configurations with DBCV score below 0.25 are skipped.
- **Best selection:** among valid configurations ($\text{DBCV} > 0.5$), the one producing the fewest clusters is chosen to favour interpretability.

3.4.2.2 Outlier Reassignment

Elements labelled as noise (cluster label = -1) by HDBSCAN are reassigned to the nearest cluster centroid using Euclidean distance:

1. Compute centroids for all valid clusters.
2. For each outlier, find the centroid with minimum Euclidean distance.
3. Assign the outlier to that cluster.
4. Recompute centroids and repeat until convergence.

3.4.3 Post-Clustering Data Enrichment

After clustering, each DOM element in the stored JSON data is annotated with its cluster assignment and the cluster colour used for visualisation. This creates the enriched data structure used by subsequent pipeline stages for similarity computation and HTML output generation.

3.5 Data Mining: Transition Matrices and Role Vectors

3.5.1 Transition Matrix Construction

For each processed page, the system extracts the sequence of cluster IDs assigned to DOM elements in traversal order. A transition matrix T is then built, where:

$$T_{i \rightarrow j} = \frac{\text{count}(i \rightarrow j)}{\sum_k \text{count}(i \rightarrow k)} \quad (3.4)$$

counts transitions between adjacent cluster IDs in the sequence.

3.5.2 Positional Statistics

For each cluster, the system computes statistics of normalised element positions within the DOM sequence:

$$\text{pos}_{norm} = \frac{\text{position}}{\max(\text{sequence_length} - 1, 1)} \quad (3.5)$$

$$\mu_{pos} = \frac{1}{n} \sum_{i=1}^n \text{pos}_{norm,i} \quad \sigma_{pos} = \sqrt{\frac{1}{n} \sum_{i=1}^n (\text{pos}_{norm,i} - \mu_{pos})^2} \quad (3.6)$$

3.5.3 Role Vector Specification

The complete role vector for each cluster combines all the above information:

$$\mathbf{R}_{cluster} = [\text{freq}, \mathbf{T}_{out}, \mathbf{T}_{in}, \mu_{pos}, \sigma_{pos}, \min_{pos}, \max_{pos}] \quad (3.7)$$

where freq is the relative occurrence frequency of that cluster across all elements, and $\mathbf{T}_{out}$, $\mathbf{T}_{in}$ are the outgoing and incoming transition probability vectors respectively.

3.6 Similarity Analysis

3.6.1 Domain-Level Aggregation

Websites are grouped by domain (via URL parsing). Role vectors within each domain are averaged to produce a domain-representative vector. Inter-domain similarity is then computed between these aggregated vectors.

3.6.2 Multi-Metric Similarity

Three complementary metrics are computed for each domain pair (A, B) :

Cosine Similarity:

$$\text{sim}_{\text{cos}}(A, B) = \frac{\mathbf{A} \cdot \mathbf{B}}{|\mathbf{A}| |\mathbf{B}|} \quad (3.8)$$

EMD Approximation (bidirectional mean minimum distance):

$$\text{sim}_{\text{EMD}}(A, B) = \frac{1}{2} [(1 - \overline{\min \text{dist}(A \rightarrow B)}) + (1 - \overline{\min \text{dist}(B \rightarrow A)})] \quad (3.9)$$

Hausdorff Approximation (bidirectional max minimum distance):

$$\text{sim}_H(A, B) = \frac{1}{2} [(1 - \max \min \text{dist}(A \rightarrow B)) + (1 - \max \min \text{dist}(B \rightarrow A))] \quad (3.10)$$

The final combined similarity uses geometric mean:

$$\text{sim}_{\text{combined}}(A, B) = (\text{sim}_{\text{cos}} \times \text{sim}_{\text{EMD}} \times \text{sim}_H)^{1/3} \times 100\% \quad (3.11)$$

3.6.3 Best Level Selection Algorithm

After computing clustering results for all requested levels, the system selects the best level using a median-based algorithm:

Algorithm: Best Clustering Level Selection

Input: clustering_results: list of {level, num_clusters, dbcv_score}

Output: best result

1. valid $\leftarrow$ results with num_clusters > 0 and dbcv_score > 0
2. **if** |valid| < 2 : **return** None
3. Sort valid by num_clusters (ascending)
4. $W \leftarrow$ num_clusters at median position of sorted list
5. filtered $\leftarrow$ results with num_clusters $\leq W$
6. **return** result with highest dbcv_score from filtered

This balances clustering quality (DBCV score) with interpretability (fewer clusters preferred).

3.7 Result Presentation

3.7.1 Colour-Coded HTML Generation

For each processed page, the system generates an HTML file in which every clustered DOM element is rendered with its cluster colour as a `background-color` CSS property. Files are organised in `output_html_files/level_N/` directories and served by the Node.js static file server on port 5005.

3.7.2 Similarity Matrix

The final output is a symmetric matrix S where $S_{ij} = S_{ji}$ gives the combined similarity score (as a percentage) between domains i and j , with $S_{ii} = 100\%$. The React frontend renders this matrix as a colour-coded heatmap using the Viridis colourmap for perceptual uniformity and accessibility.

3.8 System Parameters and Usage

Key configurable parameters:

- **Analysis levels:** any subset of 1–10; more levels increase coverage but also computation time.
- **Minimum cluster size:** set dynamically as 5–10% of total elements.
- **Processing mode:** Mode 0 processes all URLs together in a single pass; Mode 1 groups URLs by domain and processes each group separately.
- **Caching:** a TTL-based cache (`CacheManager`) stores processed URL data, allowing repeated runs to skip re-scraping already-seen URLs.

Recommended usage:

- Use levels 3–7 for general-purpose domain comparison.
- Use Mode 0 (unified processing) when all input URLs should be clustered together for a single similarity matrix.
- Monitor DBCV scores to confirm clustering quality; scores above 0.5 indicate well-separated clusters.

3.9 Methodology Summary

The S⁴D methodology provides a complete pipeline for automated web similarity analysis. Its key technical contributions are: (1) a bottom-up multi-level DOM analysis framework, (2) HDBSCAN clustering with grid-search parameter optimisation and outlier reassignment, (3) role vectors encoding frequency, transition, and positional information, (4) a geometric-mean multi-metric similarity framework, and (5) a median-based best-level selection algorithm. Together these components form an end-to-end system capable of producing interpretable cross-domain similarity results without manual parameter tuning.

Chapter 4

System Implementation

This chapter details the technical implementation of the S⁴D system, covering the backend processing engine, the Flask REST API, the Node.js static file server, and the React frontend, together with the process orchestration that ties them together.

4.1 System Architecture Overview

The S⁴D system is structured as three independent services that communicate over HTTP:

1. **Flask REST API** (port 5000) — the primary backend service that accepts analysis requests, drives the processing pipeline, and returns JSON results.
2. **Node.js static file server** (port 5005) — serves the colour-coded HTML output files generated during analysis so that the React frontend can display them in iframes.
3. **React frontend** (port 3000) — a single-page application through which users submit URLs, monitor progress, and explore results.

All three services are launched concurrently from the project root with a single `mprocs` command, which reads the process definitions from `mprocs.yaml` and multiplexes their log streams in a single terminal window.

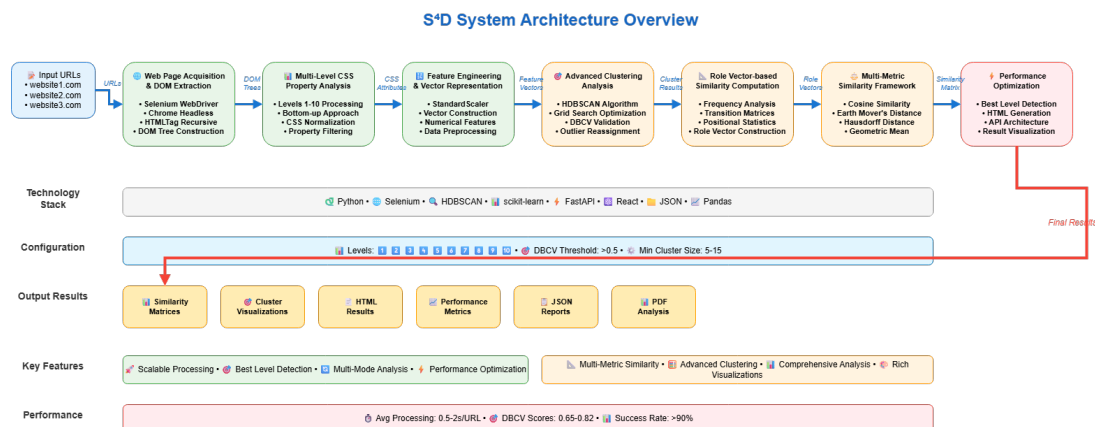


Figure 4.1: S⁴D system architecture: three independent services (Flask API, Node.js file server, React SPA) started together via `mprocs` and communicating over localhost HTTP.

4.2 Backend Implementation

4.2.1 Source Layout

The backend source tree is rooted at `backend/` and is divided into two sub-directories:

- **backend/api/** — contains `main.py` (the Flask application entry point) and `experiment_manager.py` (batch experiment orchestration).
- **backend/lib/** — contains the core processing library (`lib.py`) and the `utils/` package with the following modules:
 - **url_scrapper.py**: Selenium WebDriver automation — DOM traversal, CSS extraction, and HTML output generation.
 - **clustering.py**: HDBSCAN wrapper with grid-search parameter optimisation and DBCV validation.
 - **site_similarity.py**: multi-metric similarity computation (cosine, EMD approximation, Hausdorff approximation).
 - **DataCollector.py**: structured storage and retrieval of per-page JSON data.
 - **process.py**: pipeline orchestration coordinating the above modules.
 - **cache_manager.py**: TTL-based cache that avoids re-scraping URLs seen in recent runs.

4.2.2 Core Processing Functions

Three functions in `lib.py` form the public interface consumed by the API layer:

- **process_urls_from_api(urls, levels, mode)**: accepts a list of URLs, the requested DOM depth levels, and the processing mode (0 = unified, 1 = per-domain), runs the full pipeline, and returns a JSON-serialisable result dictionary containing clustering metadata, similarity matrices, and paths to the generated HTML files.
- **process_clusters_from_api(clusters_data)**: accepts pre-computed cluster data and recomputes the similarity matrix, useful for re-running the analysis step without repeating DOM extraction.
- **find_best_clustering_level(clustering_results)**: the median-based level selection algorithm described in Section ??.

4.3 Flask REST API

4.3.1 Server Configuration

The Flask application (`backend/api/main.py`) is initialised with `flask-cors` to allow requests from the React frontend on port 3000. An optional Swagger UI (`flasgger`) is enabled by the environment variable `ENABLE_SWAGGER=true`, providing interactive API documentation at `/apidocs`.

```

1 app = Flask(__name__)
2 CORS(app, resources={r"/*": {
3     "origins": ["http://localhost:3000"],
4     "methods": ["GET", "POST", "OPTIONS"]

```

```

5  })
6  swagger = Swagger(app) if enable_swagger else None

```

Listing 4.1: Flask application initialisation

4.3.2 API Endpoints

4.3.2.1 URL Processing

POST `/process-urls` is the primary analysis endpoint. It accepts a JSON body with three fields:

- `urls` — list of URLs to analyse.
- `levels` — list of DOM depth levels to compute (e.g., `[3, 5, 7]`).
- `mode` — processing mode: 0 for unified (all URLs in one pass) or 1 for per-domain grouping.

The handler calls `process_urls_from_api()` and returns the result as JSON, including the similarity matrix, per-level clustering summaries, and file paths for the generated HTML visualisations.

4.3.2.2 Cluster Reprocessing

POST `/process-clusters` accepts pre-existing cluster assignments and recomputes the role vectors and similarity metrics without re-running the expensive DOM extraction step.

4.3.2.3 Batch Experiments

POST `/experiments/run-batch` accepts an experiment configuration object containing one or more named test cases, each specifying its own URL list, levels, and mode. The `ExperimentManager` runs all test cases sequentially, persists results to dated sub-directories, and returns a unique experiment identifier.

4.3.2.4 Result Retrieval

- **GET** `/experiments/results/list` — returns metadata for all stored experiment runs (name, timestamp, test-case count).
- **GET** `/experiments/results/{experiment_id}` — returns the full result object for a specific run, including per-test similarity matrices and clustering summaries.

4.3.3 HTML File Proxy

GET `/output_html_files/{filename}` serves colour-coded HTML files generated during analysis. The Flask handler resolves the path relative to the project root, verifies the file exists, and streams it to the caller. This route mirrors the Node.js static server (Section ??) and allows programmatic access to individual HTML snapshots via the API base URL.

4.4 Node.js Static File Server

A lightweight Node.js/Express server (`backend/server/server.js`) listens on port 5005 and mounts the `output_html_files/` directory as a static asset path:

```

1  const express = require('express');
2  const app = express();
3  const outputDir = path.join(__dirname, '../api/output_html_files');
4  app.use('/output_html_files', express.static(outputDir));
5  app.listen(5005, () => console.log('Server is running on port 5005'));

```

Listing 4.2: Node.js static server

The React frontend embeds the resulting URLs (e.g., `http://localhost:5005/output_html_files/level_3/example`) in iframes to render the colour-coded page visualisations alongside the similarity matrix.

4.5 React Frontend

4.5.1 Application Structure

The frontend is a Create-React-App single-page application. All user-facing logic is contained in a single component, `src/components/URLChipForm.js`, which manages:

- **URL input:** a multi-value chip input that validates and lists the submitted URLs.
- **Parameter selection:** checkboxes for DOM depth levels and a toggle for processing mode.
- **Progress monitoring:** polling the Flask API during long-running analyses and displaying status messages.
- **Result display:** rendering the similarity matrix as a colour-coded heatmap (Viridis colourmap) and embedding the HTML output files via the Node.js server URLs.

4.5.2 Communication with Backend Services

All API calls use the browser’s `fetch` API targeting the Flask server at `http://localhost:5000`. The Node.js server at `http://localhost:5005` is used exclusively for `<iframe>` sources that load the generated HTML output files.

4.6 Process Orchestration

The `mprocs.yaml` file at the project root defines three process entries:

- **api:** `python backend/api/main.py` — starts the Flask server on port 5000.
- **server:** `node backend/server/server.js` — starts the Node.js file server on port 5005.
- **frontend:** `npm start` (in `frontend/`) — starts the React development server on port 3000.

Running `mprocs` from the project root launches all three concurrently, with each service’s output displayed in a separate pane. This eliminates the need for multiple terminal sessions during development and testing.



Figure 4.2: React interface: URL chip input form, level/mode selectors, progress feedback, and the resulting similarity heatmap with linked HTML visualisations.

4.7 Caching

The `CacheManager` class (`backend/lib/utils/cache_manager.py`) implements a file-based TTL cache. When a URL is submitted for processing, the system checks whether a recent (non-expired) result already exists in the cache directory. If found, the cached JSON data is loaded directly, skipping DOM extraction and clustering. This significantly reduces processing time when running repeated experiments on the same URL sets. Cache expiry and storage paths are configurable via `backend/lib/config.json`.

4.8 Implementation Summary

The S⁴D implementation is organised as three cooperating services: a Flask REST API that drives the analysis pipeline, a Node.js static server that delivers HTML visualisations, and a React SPA that provides the user interface. The `mprocs` tool starts all three with a single command. The backend processing library exposes three well-defined entry points consumed by the API layer, and the TTL cache reduces redundant computation across repeated runs. This modular separation ensures that each component can be tested and maintained independently while composing into a coherent end-to-end system.